

QuestMaker Technical Design Document

Project Information

Version:Unity 6000.4.5f1

Language: C#

Key package | com.unity.graphtoolkit 0.4.0-exp.

Version control: GitHub

1. Code style & project structure

1.1 Assembly dependencies

Editor -> Runtime

Editor -> Domain

Runtime -> Domain

Runtime -> TextMeshPro

1.2 Folder structure

All custom made content lives under Assets/QuestMaker

Assets/QuestMaker/

— Docs/	
— Prefabs/	
— QuestGraphs/	.qmgraph files
— Resources/Quests/	compiled QuestSO
— Scenes/	demo
— ScriptableObjects/	Item assets
— Scripts/	
— Data/	QuestMaker.Domain
— Editor/	QuestMakerEditor
— Runtime/	QuestMaker.Runtime

1.3 Naming conventions

Public types, methods, properties -> *PascalCase*: QuestManager.

Class-level fields -> *start with underscore, camelCase*: _exp, _loadingPath.

Interfaces -> *`I` prefix*: IGameReference.

Constants -> *UPPER_SNAKE*: BLOCK_NODE_OPTION.

Editor node classes -> *`QM` prefix*: QMBaseNode, QMStartingNode.

Events -> *`On` + past tense*: OnItemCollected.

Fire events -> *`Fire` prefix methods*: FireNpcTalked.

Namespaces -> *mirror the assembly*: QuestMaker.Domain.

One name per feature across layers -> CollectStepData (data) / CollectQuestStep (runtime) / CollectStepBlockNode (editor)

2. Flow and data models

2.1 Pipeline for quests

Flow Chart

Graph -> Compiler -> Asset -> Manager/loader -> Progress -> Completion

2.2 Data model

QuestSO:

QuestSO -> ObjectiveData : objectives (ordered)

QuestSO -> PrerequisiteData

QuestSO -> RewardData

QuestSO -> SpecialEventData

QuestSO -> Description

Abstract QuestStepData

QuestStepData -> Factory method for runtime steps (abstract QuestStep)

CollectStepData : QuestStepData

SlayStepData: QuestStepData

...

Etc. for all quest Step Types.

Abstract QuestStep : IRuntimeStep

QuestStep -> Progress Text.

QuestStep -> isComplete.

QuestStep -> Events: Started, Changed, Finished.

CollectQuestStep: QuestStep

SlayQuestStep: QuestStep

...

Etc. for all quest step types

2.3 Compilation

Flow Chart

StartingNode -> Context Output Port

StartingNode -> Objective Output Port

ObjectiveNode -> Next Objective Output Port

ContextNodes -> Module Builders via Registry.

ObjectiveNodes -> Objective Builders via Registry.

Registry -> Loops all builders to write on QuestSO.

Builders : IQuestModuleBuilder + individual modules

RewardModule : IQuestModuleBuilder, IExpModule, IItemModule, IAbilityModule, IReputationModule

Nodes and modules are auto-discovered, no need for registration

2.4 Quest lifecycle

MISSING_REQUIREMENTS: By default on loading, Checks Prerequisites.
MISSING_REQUIREMENTS -> CAN_START: After Prerequisites are met.
CAN_START -> IN_PROGRESS: After accepting a quest on Quest Giver.
IN_PROGRESS --> CAN_FINISH : All objectives done
CAN_FINISH --> COMPLETED : turned in

2.5 Runtime Quest flow

QuestManager -> Loads quest via Quest Loader.
QuestManager -> Creates Runtime Quest.cs
Quest.cs -> Creates runtime objectives/steps.
Steps -> Sub to events.
QuestManager -> Chains quests where needed.
QuestManager -> Checks Prerequisites.
QuestGiver -> Gets assigned its appropriate quests (turn-ins, hand-ins).
Player interacts with QuestGiver -> QuestGiver calls Manager TryStartQuest().
Player takes quest related action -> Event Fires.
Event Fires -> Steps update progress.
All objectives completed -> Fires Quest lifecycle Event: can be completed.
Quest Lifecycle Event Fired -> QuestGiver Indicators updated.
Player interacts with QuestGiver for completion -> QuestGiver calls Manager TryFinishQuest()
Quest successfully completes -> Rewards given.